BÀI 10: REFRESH TOKEN TRONG NESTJS

Mục tiêu bài 10: Bài này sẽ giới thiệu đến các bạn về Refresh Token trong Nestjs, giới thiệu về qui trình hoạt động và cách tạo Refresh Token như thế nào trong Nestjs

10.1. KHÁI NIỆM REFRESH TOKEN

Trong **NestJS**, khi triển khai xác thực người dùng với **JWT** (JSON Web Token), chúng ta thường sử dụng **Access Token** để xác thực người dùng trong thời gian ngắn. Tuy nhiên, do **Access Token** thường có thời hạn ngắn (ví dụ 15 phút, 1 giờ) để giảm nguy cơ bị đánh cắp, nên cần có **Refresh Token** để tự động cấp lại **Access Token** mới mà không cần đăng nhập lại.

Như vậy, **Refresh Token** là:

- Là một chuỗi **token** dài hạn được cấp cùng với **Access Token**.
- Được lưu trữ an toàn (thường là trong **httpOnly cookie** hoặc **local storage**) và gửi lên để yêu cầu cấp lại **Access Token** khi **token** hết hạn.
- Có thời gian sống dài hơn **Access Token** (ví dụ: 7 ngày, 30 ngày...).

10.2. QUI TRÌNH HOẠT ĐỘNG VỚI REFRESH TOKEN TRONG NESTJS

Bước 1: Người dùng đăng nhập

- Hệ thống tạo ra:
    - access_token (ví dụ sống 15 phút)
    - refresh_token (ví dụ sống 7 ngày)
- Trả về cho client và lưu trữ an toàn.

Bước 2: Gọi API với Access Token

- Trong các request sau đó, client dùng access_token để truy cập tài nguyên.

Bước 3: Access Token hết hạn

- Khi Access Token hết hạn, client gửi refresh_token đến một endpoint như /auth/refresh.

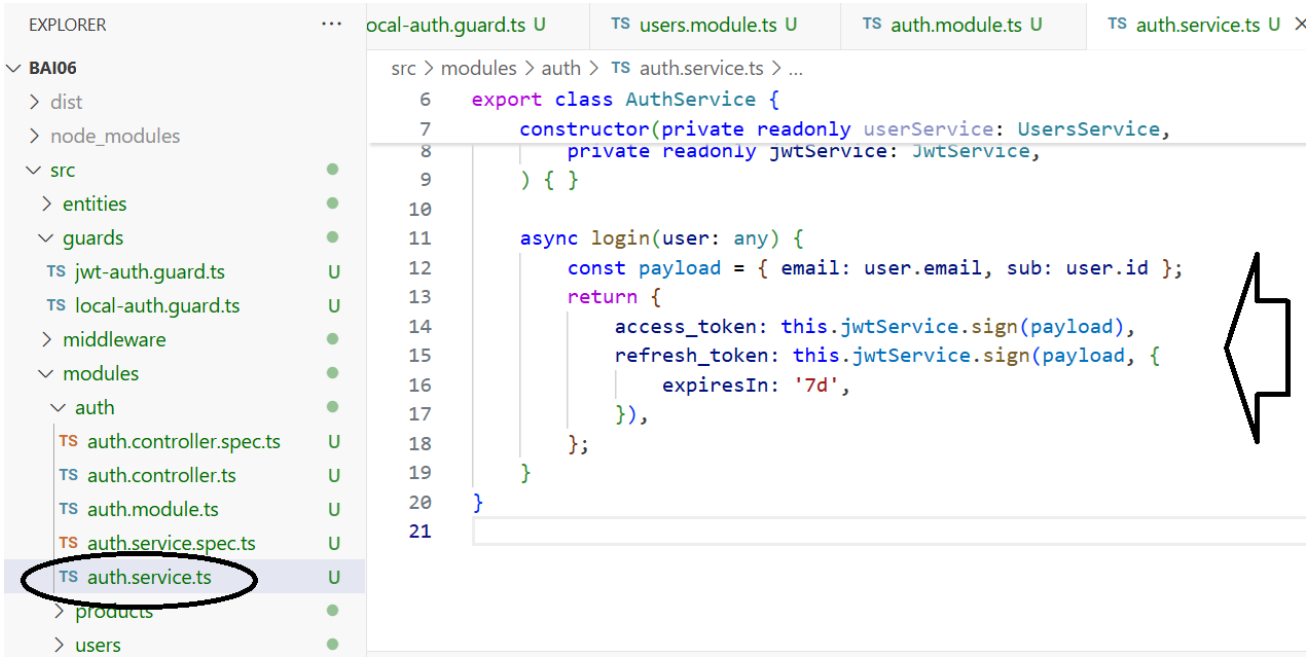
Bước 4: Cấp lại Access Token

- Backend kiểm tra refresh_token hợp lệ không:
- Nếu hợp lệ → cấp lại access_token mới.
- Nếu không hợp lệ → yêu cầu đăng nhập lại.

10.3. TẠO REFRESH TOKEN TRONG NESTJS

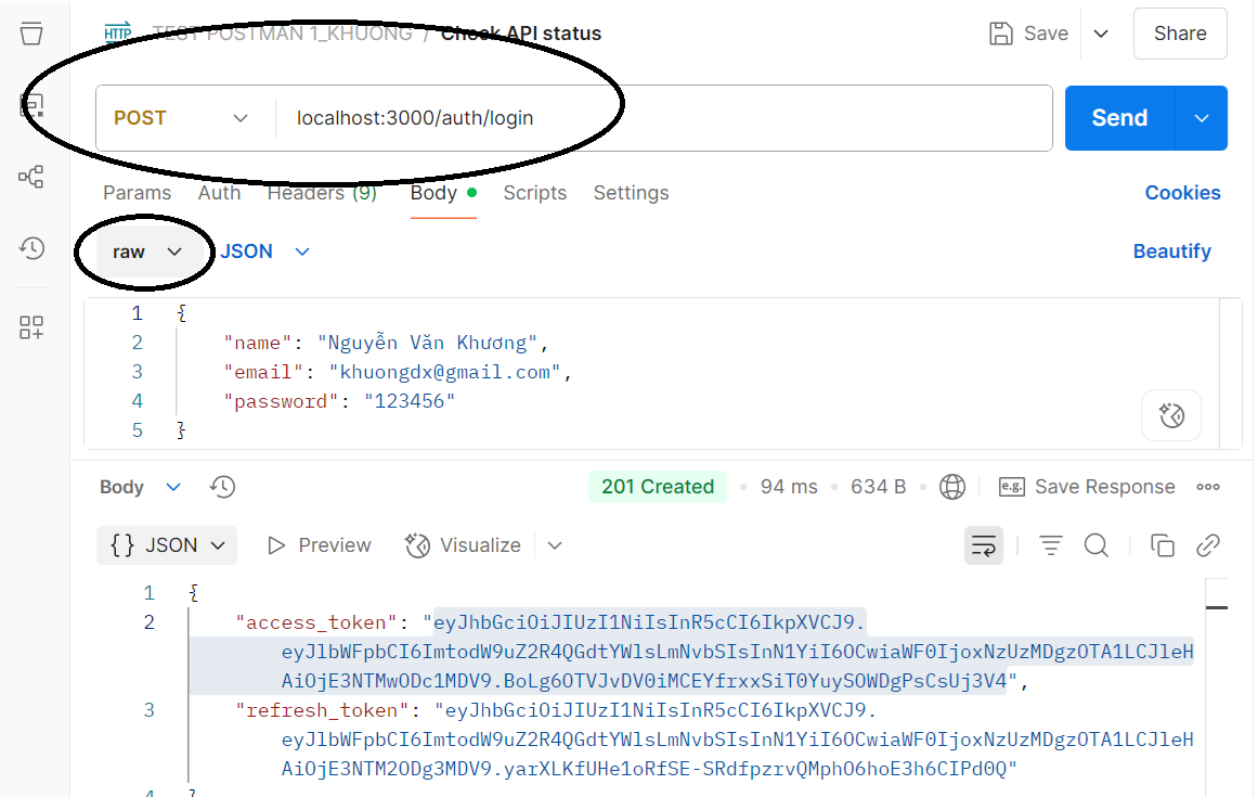
- Các bạn mở lại dự án ở bài trước trong VSCode

- Tiếp tục các bạn mở tập tin auth.service.ts để chỉnh sửa lại hàm login() để thêm refresh_token như bên dưới:

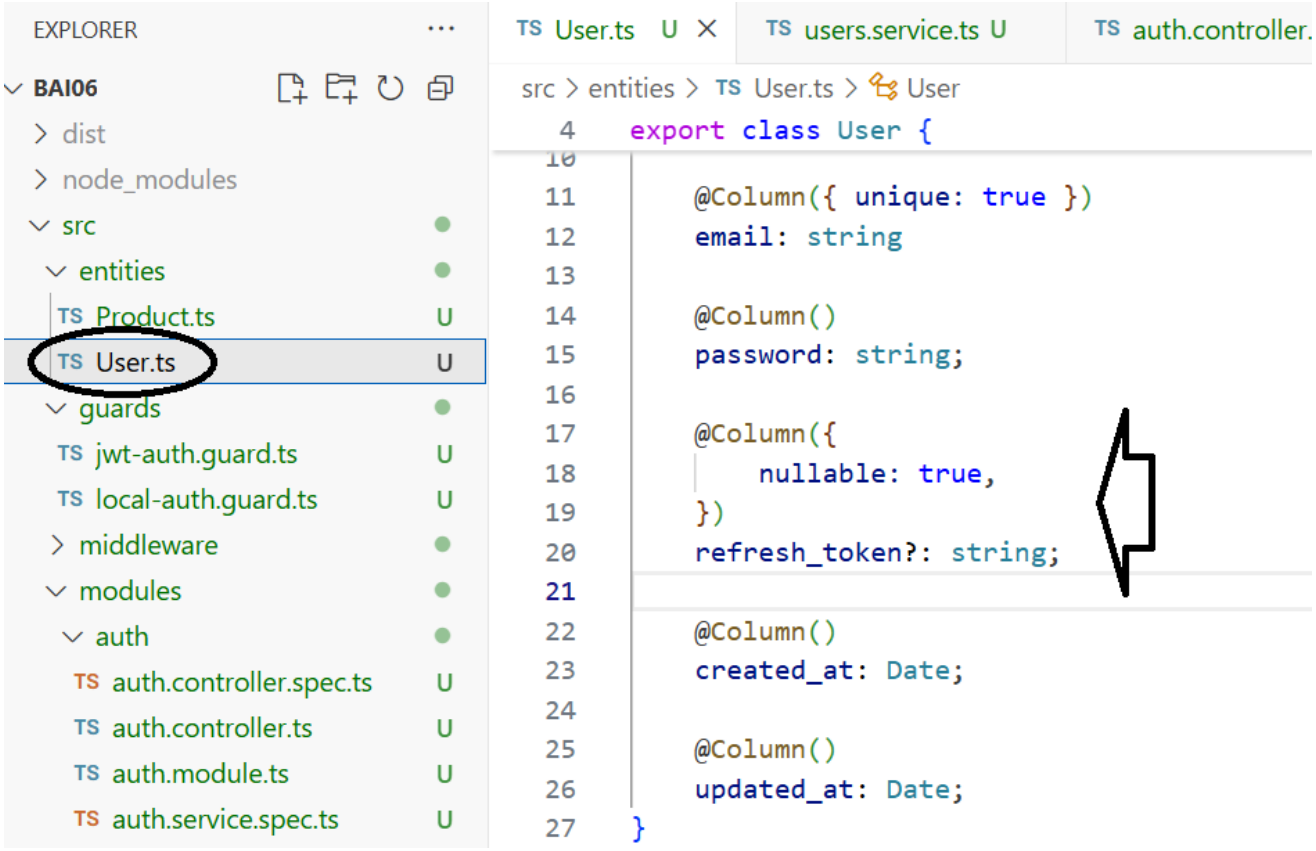


- Biên dịch lại chương trình: **npm run start:dev**

- Mở chương trình **Postman** thiết lập như bên dưới sẽ thấy hiển thị thêm một **refresh_token**:



- Tiếp tục chúng ta thực hiện lưu **refresh_token** vào cơ sở dữ liệu bằng cách: mở tập tin **User.ts** viết thêm đoạn code như bên dưới:



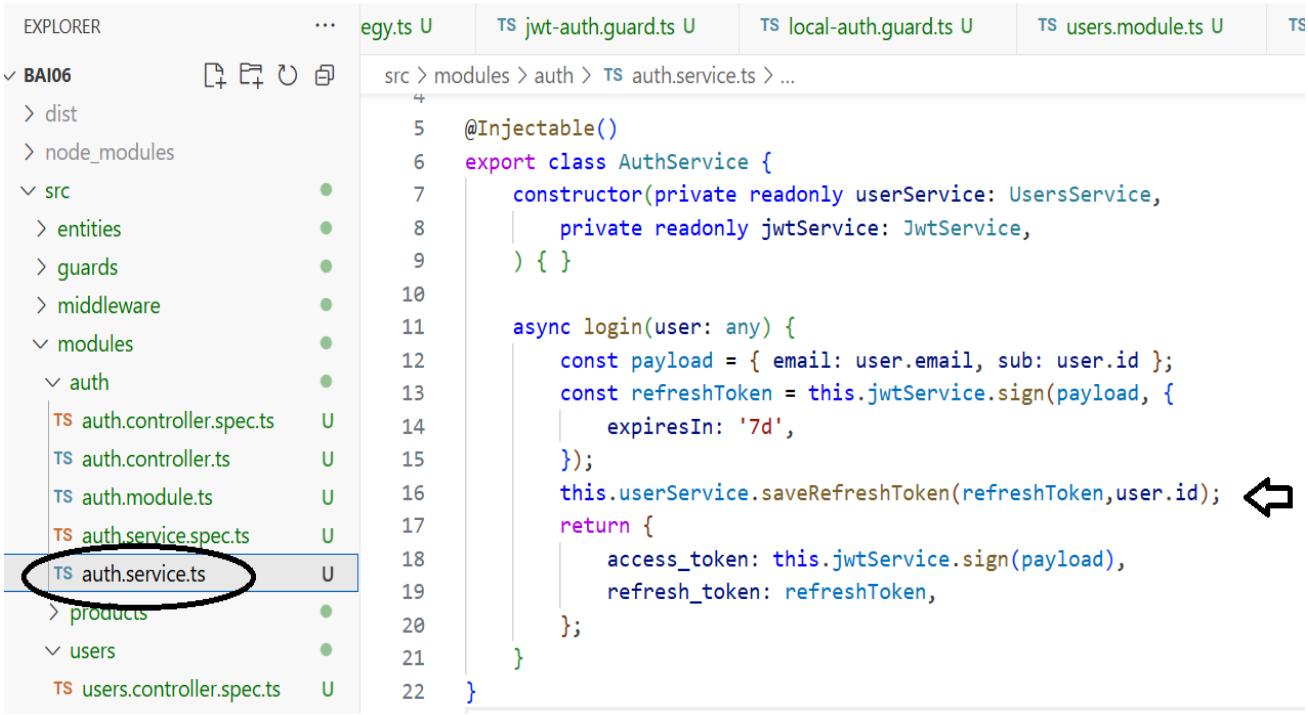
- Tiếp theo các bạn mở tập tin **auth.service.ts** chỉnh sửa code hàm **login()** lại như sau:



- Tiếp tục các bạn **users.service.ts** và viết thêm hàm **saveRefreshToken ()** như bên dưới:

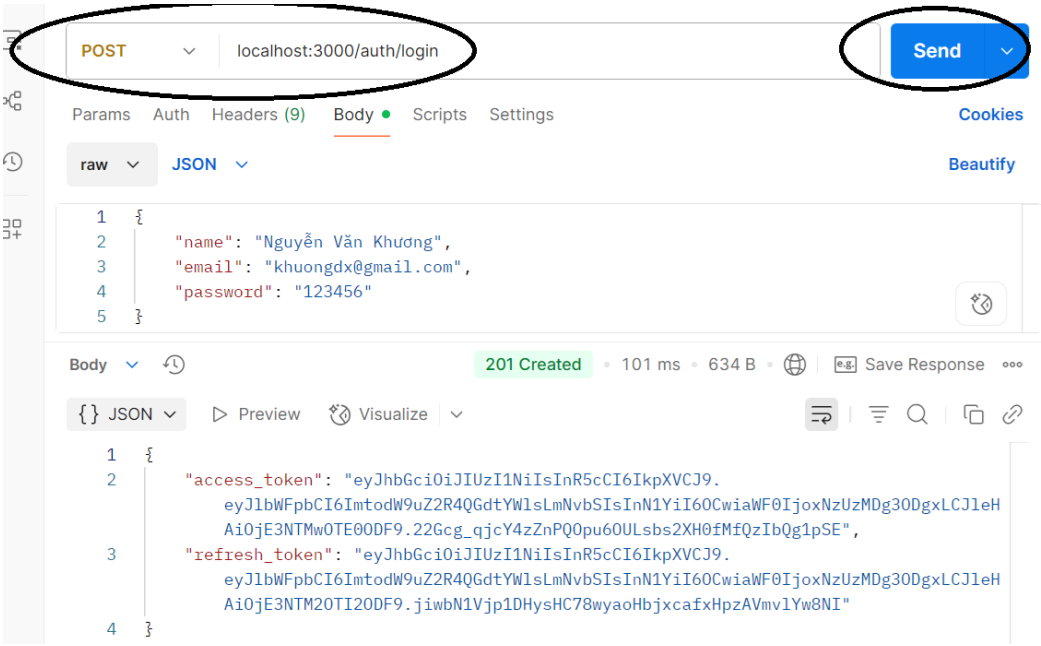


- Tiếp tục mở lại tập tin **auth.service.ts** thêm dòng lệnh để gọi hàm **saveRefreshToken ()** vừa tạo như bên dưới:

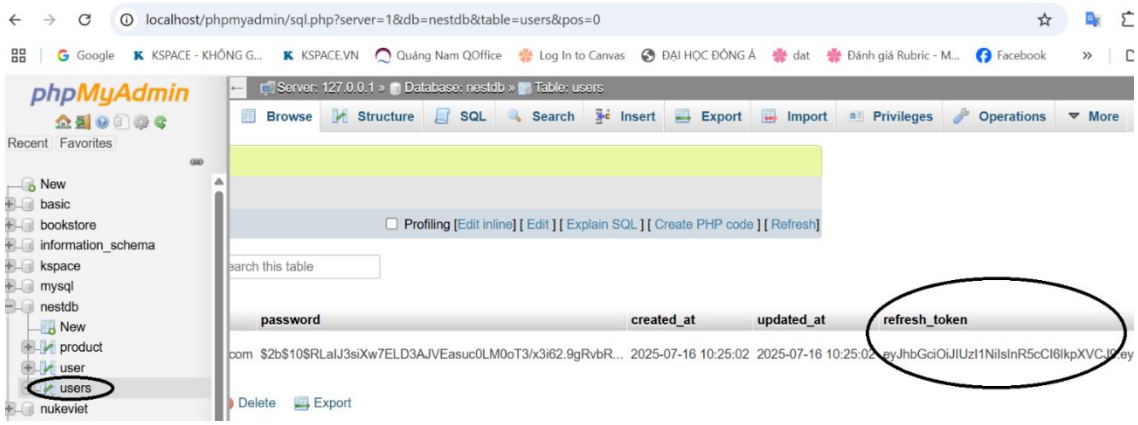


- Biên dịch lại chương trình: **npm run start:dev**

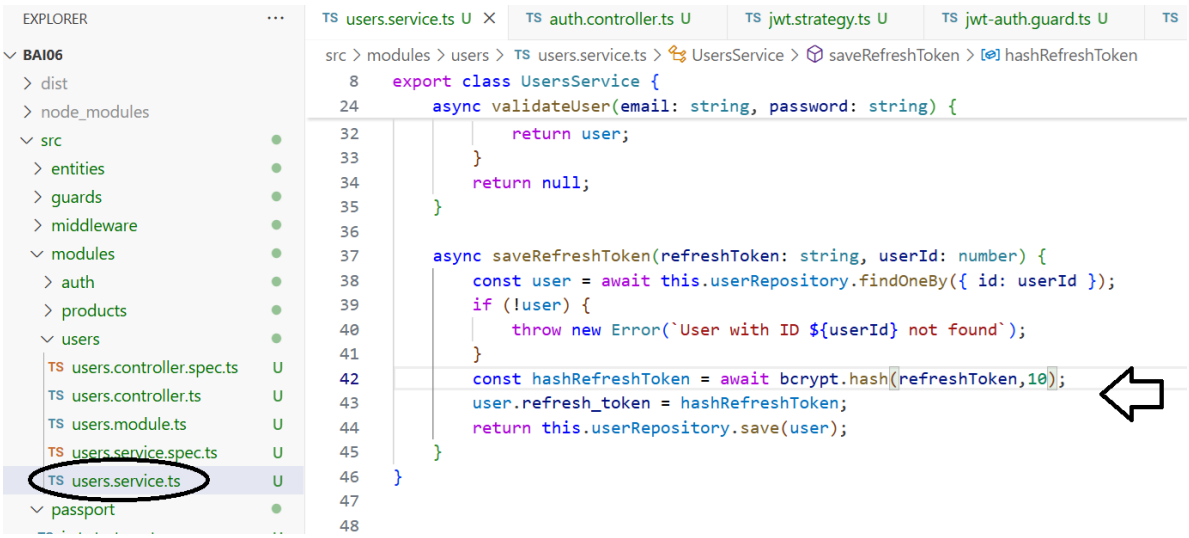
- Mở chương trình **Postman** thiết lập như bên dưới, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/auth/login**



- Mở cơ sở dữ liệu **Mysql** ra sẽ thấy kết quả **refresh_token** đã được lưu vào cơ sở dữ liệu:

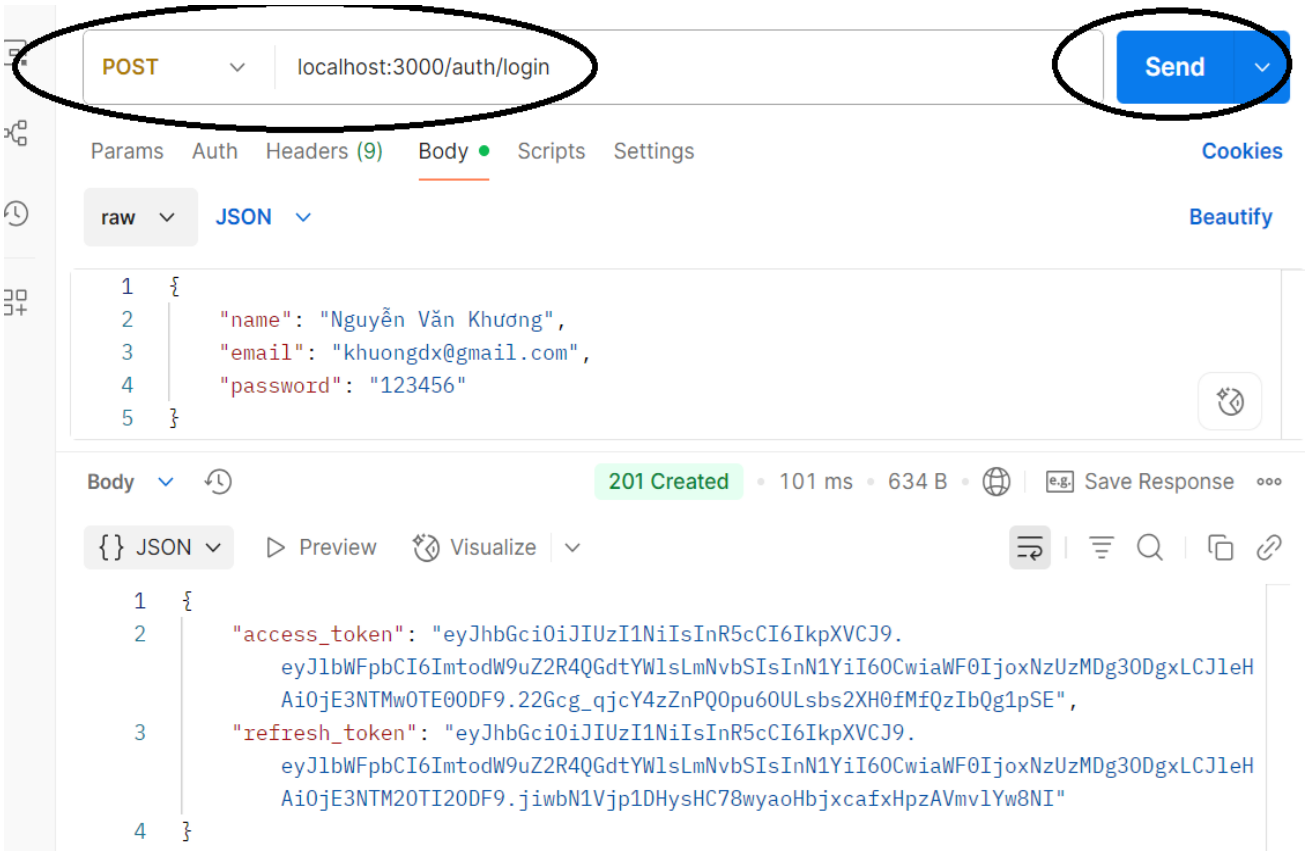


- Tuy nhiên việc lưu **refresh_token** vào cơ sở dữ liệu như thế này sẽ rất dễ bị lộ và không an toàn nên trước khi lưu chúng ta thực hiện mã hóa **refresh_token** bằng cách mở lại tập tin **users.service.ts** và sửa lại code như sau:

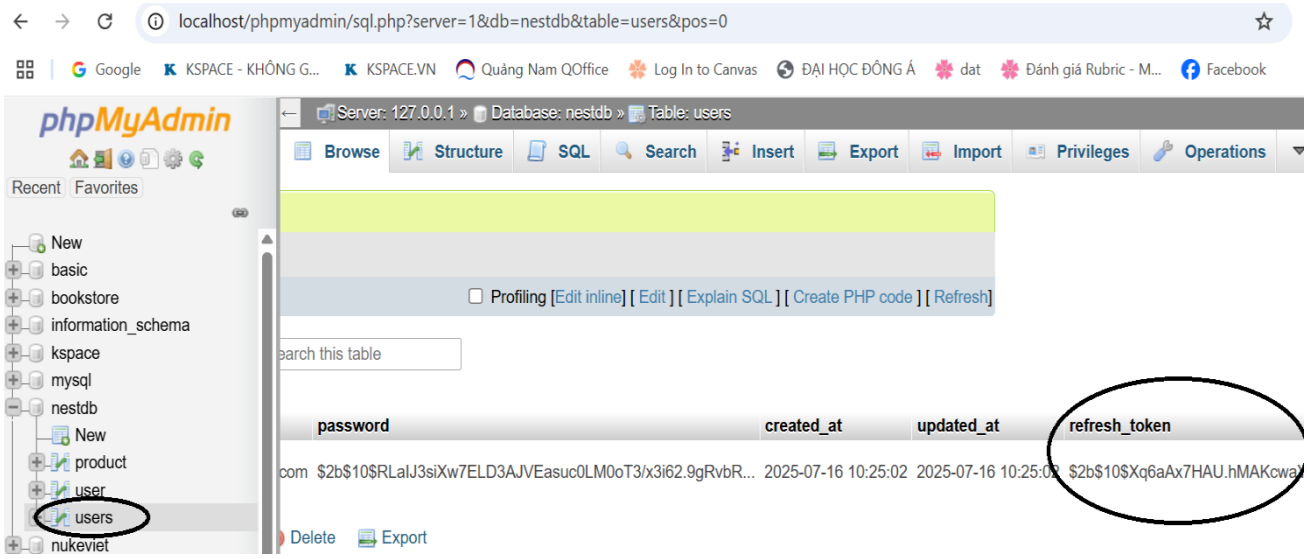


- Biên dịch lại chương trình: **npm run start:dev**

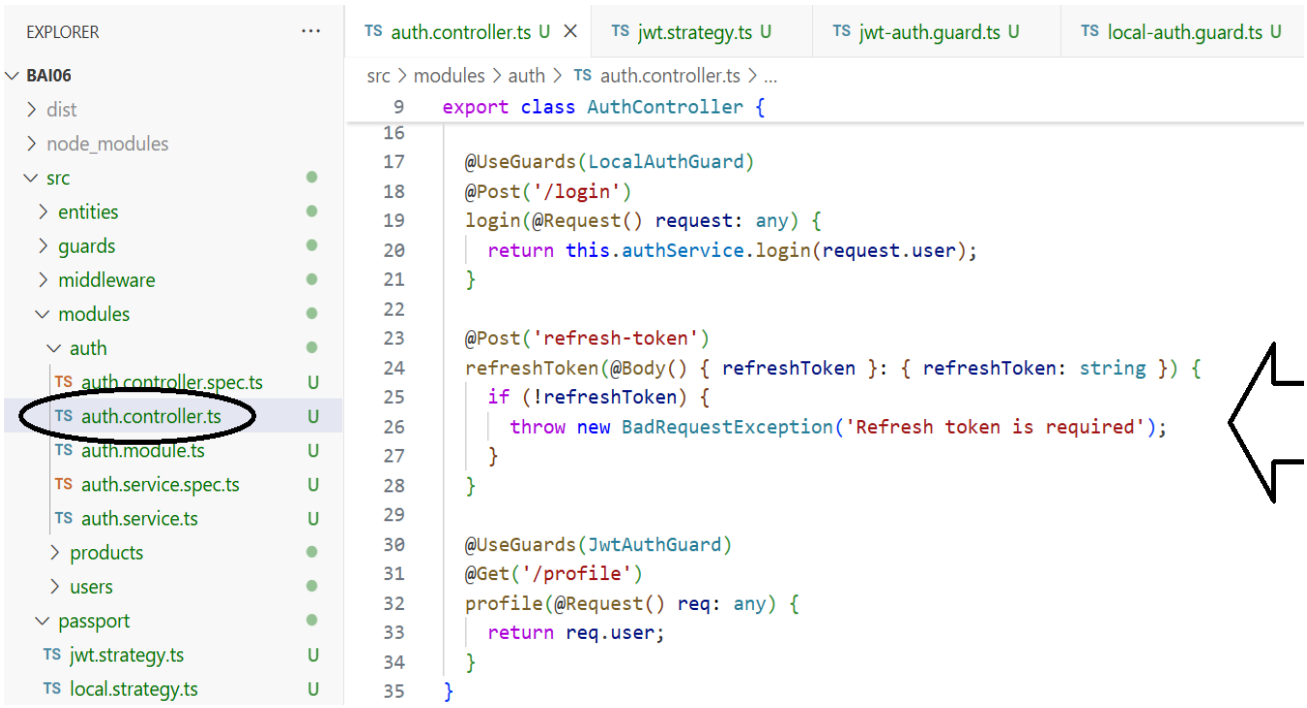
- Mở chương trình **Postman** thiết lập như bên dưới, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/auth/login**



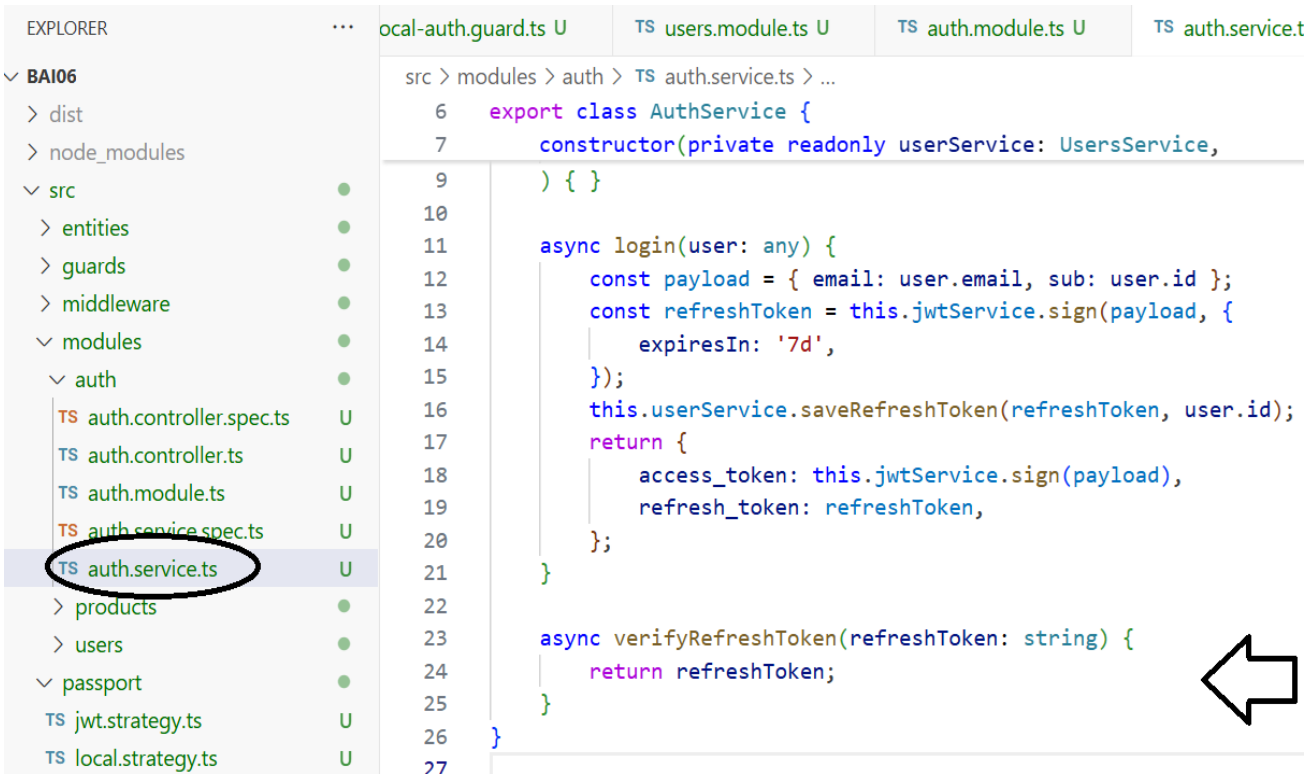
- Mở cơ sở dữ liệu **Mysql** ra sẽ thấy kết quả **refresh_token** đã **được mã hóa** và lưu vào cơ sở dữ liệu:



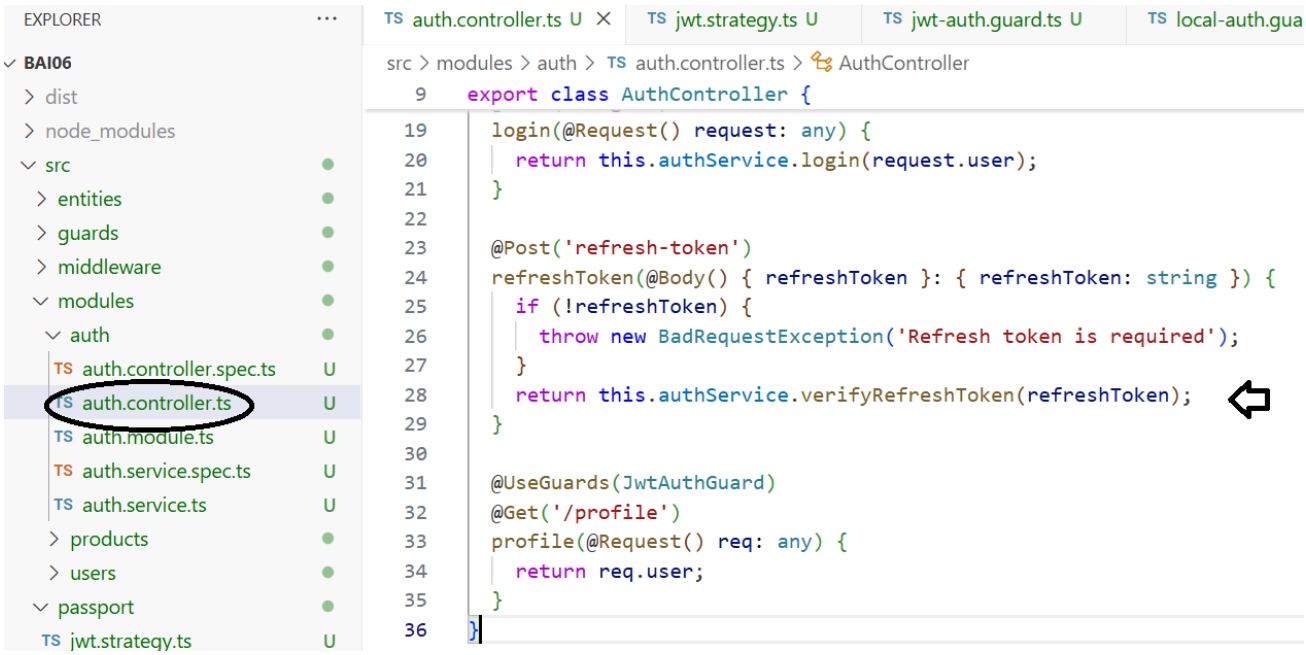
- Tiếp tục chúng ta tạo thêm một **API** bằng cách mở tập tin **auth.controller.ts** và viết thêm code như sau:



- Tiếp tục các bạn thêm một **Service** bằng cách mở tập tin **auth.service.ts** và viết thêm đoạn code sau:

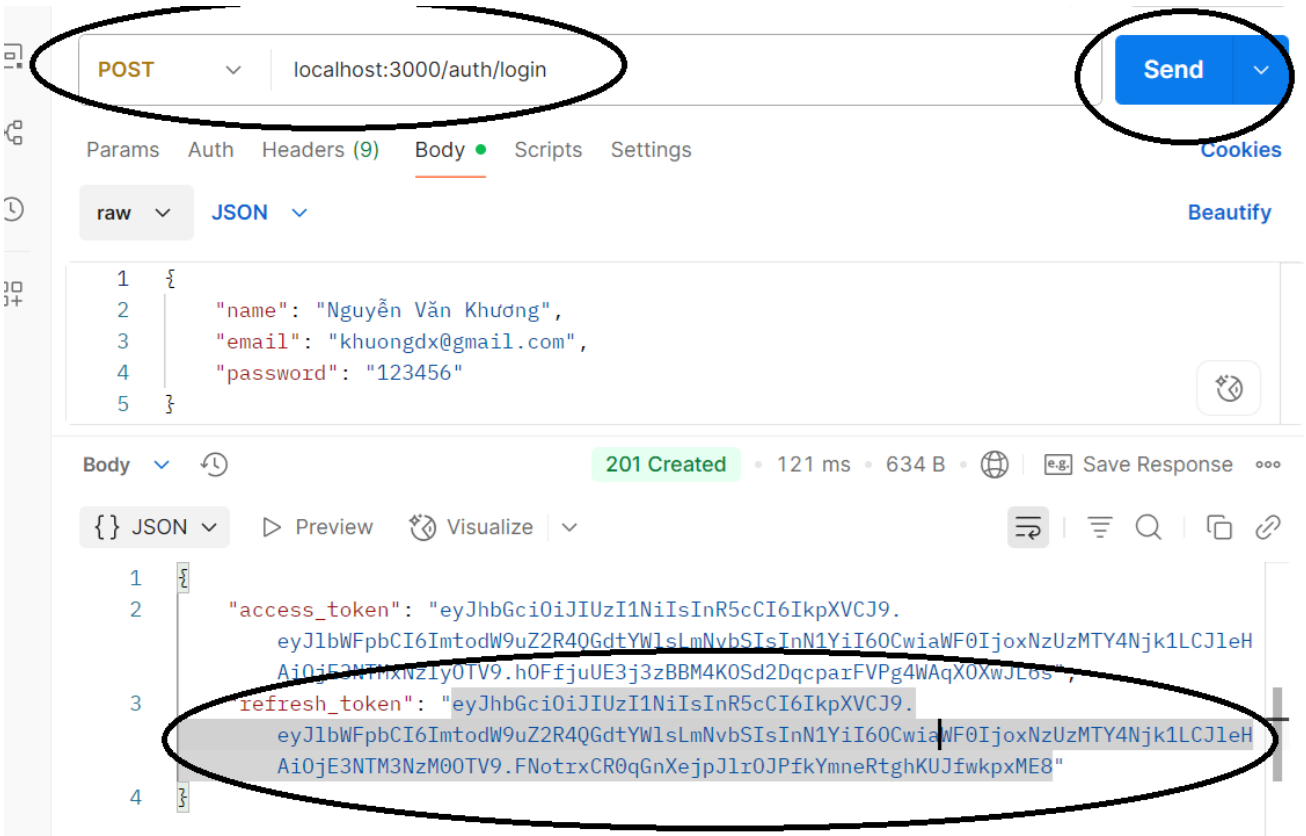


- Bây giờ các bạn mở tập tin **auth.controller.ts** và viết thêm dòng code như sau:

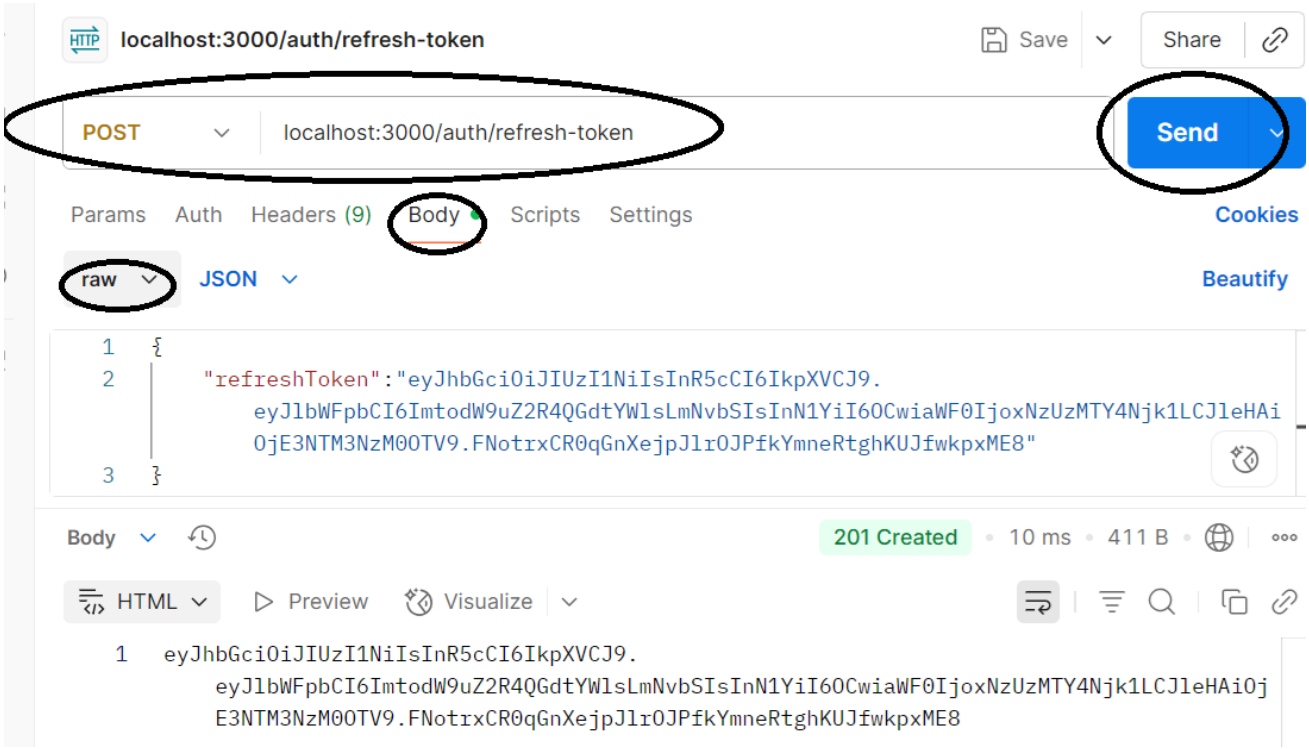


- Biên dịch lại chương trình: **npm run start:dev**

- Mở chương trình **Postman** thiết lập như bên dưới để kiểm tra, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/auth/login** và chọn **Send**

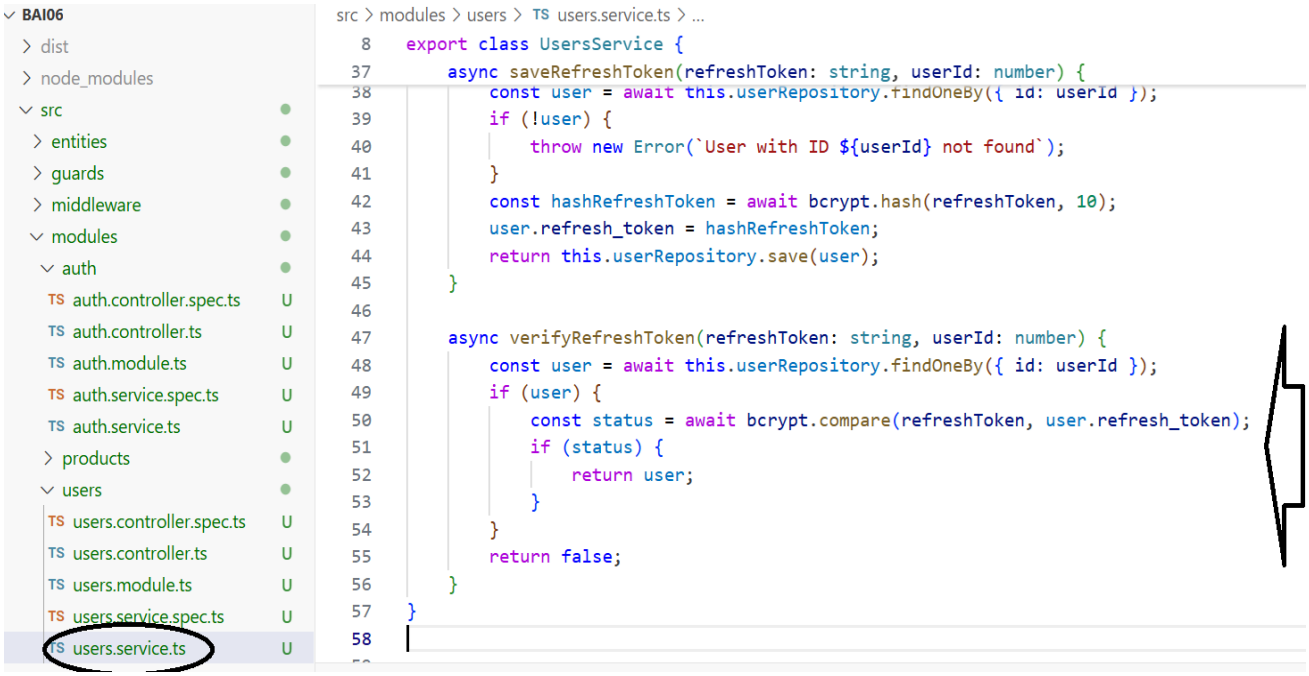


- Các bạn **copy** dữ liệu của **refresh_token** và mở một tab mới trong chương trình **Postman**, chọn phương thức **POST**, gõ đường dẫn**: localhost:3000/auth/refresh-token**, chọn **Body**, chọn **Raw**, gõ nội dung vào **Raw** với mã các bạn dán mã dữ liệu của **refresh_token** đã **copy**.



- Chọn **Send** sẽ thấy trả về **RefreshToken**.

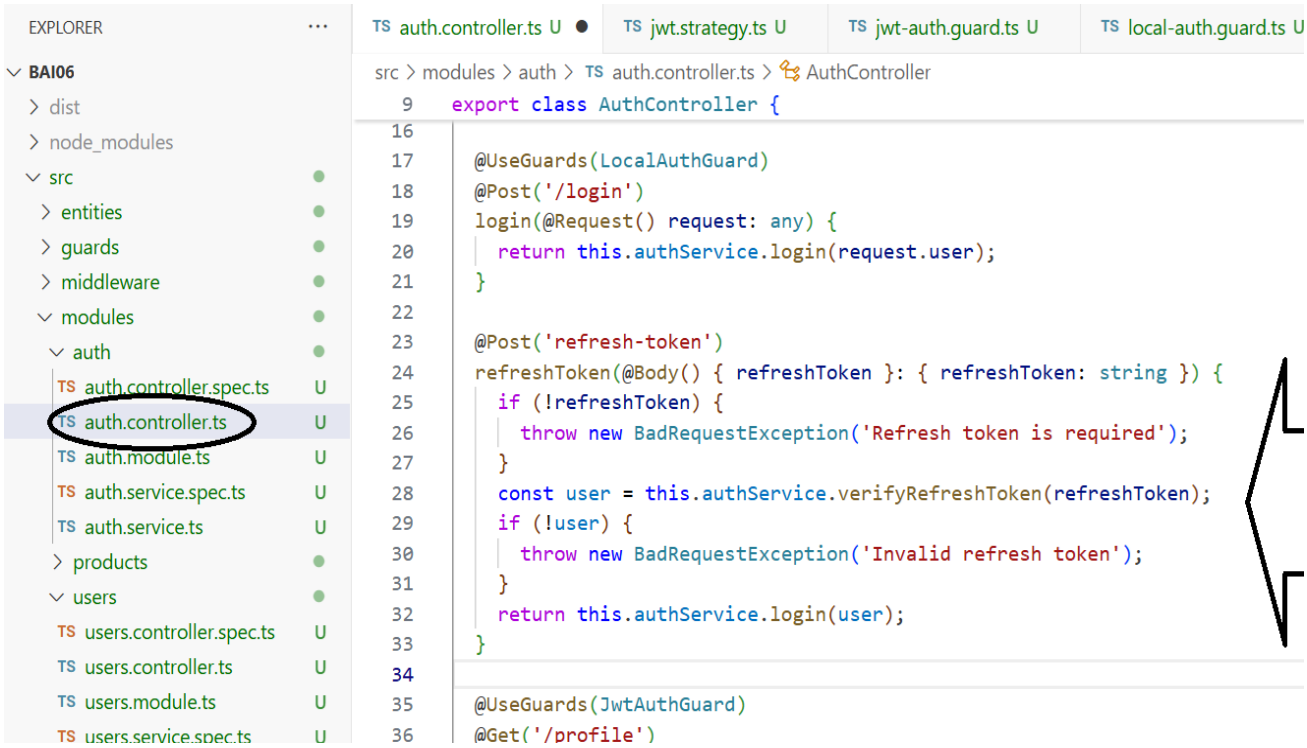
- Tiếp tục, chúng ta thực hiện **verify refreshToken** bằng cách mở tập tin **users.service.ts** và viết thêm đoạn code như sau:



- Tiếp tục mở tập tin **auth.service.ts** và sửa lại đoạn code hàm **verifyRefreshToken ()** như sau:

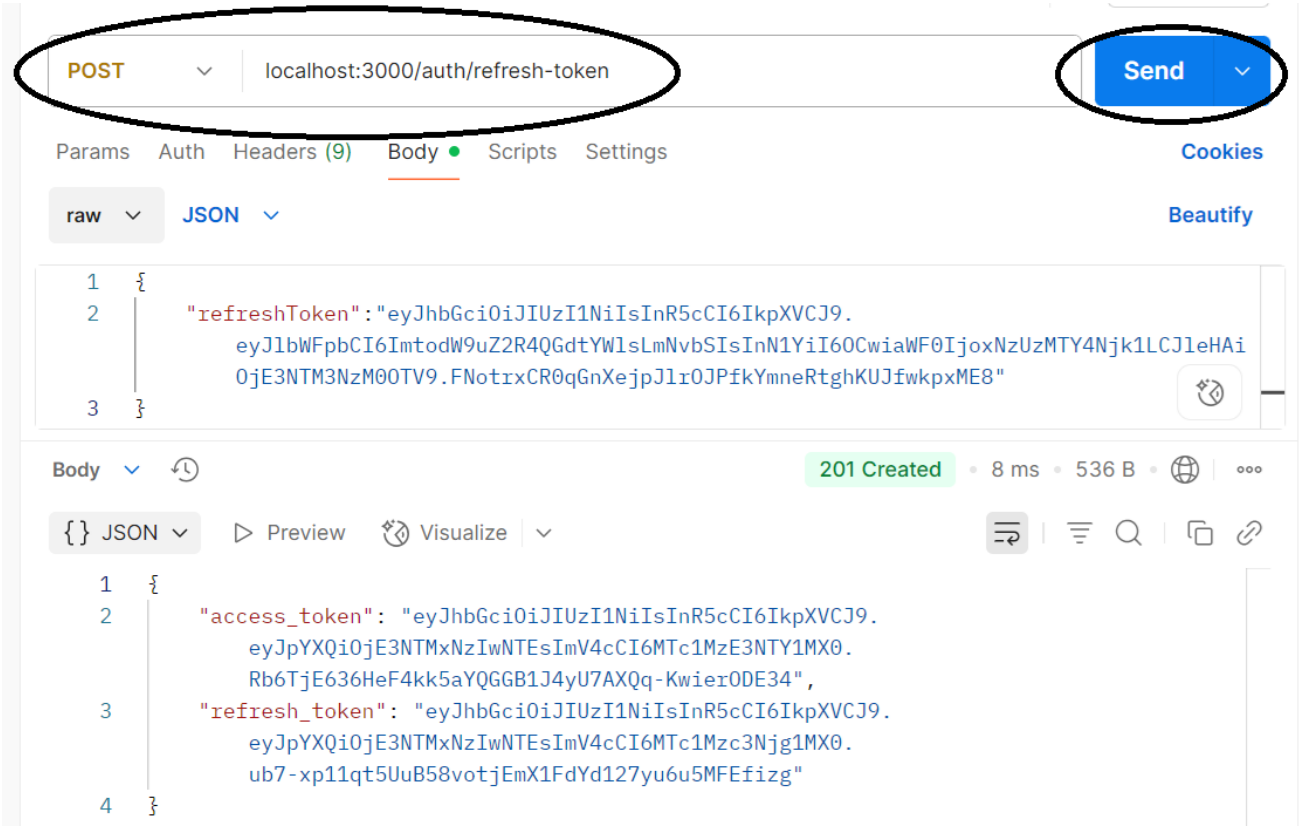


- Tiếp tục mở tập tin **auth.controller.ts** và sửa lại **code** cho hàm **refreshToken()** như bên dưới:



- Biên dịch lại chương trình: **npm run start:dev**

- Mở chương trình **Postman** thiết lập như bên dưới để kiểm tra, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/auth/refresh-token** và chọn **Send** sẽ thấy tạo được **access_token, refresh_token** mới nếu xác thực đúng.


POST
localhost:3000/auth/refresh-token
Send
Params
Auth
Headers (9)
Body
Scripts
Settings
Cookies
raw
JSON
Beautify
{
"refreshToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJlbWFpbCI6ImtodW9uZ2R4QGdtYWlsLmNvbSIsInN1YiI6OCwiaWF0IjoxNzUzMTY4Njk1LCJleHAi
OjE3NTM3NzM0OTV9.FNotrxCR0qGnXejpJlrOJPfkYmneRtghKUJfwkpxME8"
}
Body
201 Created
8 ms
536 B
JSON
Preview
Visualize
{
"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpYXQiOjE3NTMxNzIwNTEsImV4cCI6MTc1MzE3NTY1MX0.
Rb6TjE636HeF4kk5aYQGGB1J4yU7AXQq-KwierODE34",
"refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpYXQiOjE3NTMxNzIwNTEsImV4cCI6MTc1Mzc3Njg1MX0.
ub7-xp11qt5UuB58votjEmX1FdYd127yu6u5MFEfizg"
}